

# c sdd

*by* Rab Nawaz

---

**Submission date:** 06-Jan-2026 04:29AM (UTC-0800)

**Submission ID:** 2853193446

**File name:** counselX\_sdd.pdf (1.38M)

**Word count:** 3004

**Character count:** 20293



<sup>1</sup>  
**COMSATS University Islamabad**  
**Vehari Campus**

# **SOFTWARE DESIGN DESCRIPTION**

**(SDD Document)**

*for*

**Counsel-X**

**Version 1.0**

By

Asad Naeem      CIIT/FA22-BCS-013

Samran Zafar      CIIT/FA22-BCS-044

<sup>1</sup>

***Supervisor***

**Dr. Umar Rashid**

*Bachelor of Science in Computer Science (2022-2026)*

## Table of Contents

|                                                        |     |
|--------------------------------------------------------|-----|
| Revision History .....                                 | iii |
| Application Evolution History .....                    | iv  |
| 2. Introduction.....                                   | 1   |
| 2. Design Methodology and Software Process Model:..... | 1   |
| 3. System Overview .....                               | 2   |
| 3.1 Architectural Design .....                         | 3   |
| 3.2 Process flow/Representation: .....                 | 4   |
| 4. Design models.....                                  | 8   |
| 4.1 Class Diagram .....                                | 8   |
| 4.2 Sequence Diagram.....                              | 9   |
| 4.3 Data Flow Diagram (DFD Level 0–2) .....            | 10  |
| 1. 5. Data Design.....                                 | 12  |
| 5.1 Data Dictionary .....                              | 12  |
| 6. Algorithm & Implementation .....                    | 14  |
| 7. Software Requirements Traceability Matrix.....      | 18  |
| 1. 7.1 Traceability Matrix: .....                      | 19  |
| 8. Human Interface Design .....                        | 20  |
| 8.2 Screen Objects and Actions: .....                  | 23  |
| 9. Appendix.....                                       | 24  |

**Revision History**

| Name | Date | Reason for changes | Version |
|------|------|--------------------|---------|
|      |      |                    |         |
|      |      |                    |         |

### Application Evolution History

| Comments (by committee)<br>*include the ones given at scope time both in doc and presentation | Action Taken |
|-----------------------------------------------------------------------------------------------|--------------|
|                                                                                               |              |
|                                                                                               |              |

Supervised by  
**Dr. Umar Rashid**

Signature\_\_\_\_\_

## 1. Introduction

The Counsel-X project is a web-based platform designed to offer students easy access to academic counselling, career guidance, and mental health support through real-time interaction with counsellors. By addressing the limitations of traditional in-person counselling, the platform eliminates geographical and time barriers. Built with modern technologies like React, Node.js, and MongoDB, Counsel-X ensures scalability, performance, and a smooth user experience. Key features include secure user authentication, live chat for counselling sessions, appointment scheduling, a resource library, and dedicated dashboards for students, counsellors, and administrators, making it easier for students to access the support they need efficiently.

1

## 2. Design Methodology and Software Process Model:

### Design Methodology:

The Object-Oriented Programming (OOP) methodology has been chosen for this project. OOP is well-suited for creating scalable and maintainable systems, as it allows for modular development and clear organization of code. By breaking the system into objects like User, Counsellors, Session, and Appointment, the system can be easily extended and updated without affecting the entire platform. Each object can represent real-world entities and their interactions, making the development process more intuitive and flexible..

2

### Software Process Model:

The Agile software process methodology will be used to make Counsel-X. The Agile technique supports incremental development, which gives stakeholders more freedom and lets them give input all the time. This is the best way to go about things because the project has different sorts of users and features that may change dependent on what they require. The project will be broken up into smaller projects, or sections, each of which will focus on a different feature, such as user authentication, scheduling, or the chat system. After each sprint, progress will be looked at and improvements will be made depending on what people say. This procedure of repeating steps makes sure that the final product fits the needs of the consumer and can alter as development goes on.

### 3. System Overview

The Counsel-X platform is a web-based system designed to provide students with easy and accessible counselling services for academic, career, and mental health support. The system aims to overcome traditional counselling barriers such as geographical limitations and scheduling conflicts by enabling real-time communication between students and counsellors. The platform supports features such as appointment scheduling, live chat sessions, secure payment processing, and resource access.

This system consists of several modules:

- **User Authentication:** Handles secure user login and profile management for students, counsellors, and administrators.
- **Live Chat Module:** Allows students and counsellors to communicate in real-time during scheduled sessions.
- **Appointment Scheduling:** Students can make appointments for counselling when counsellors have free time.
- **Resource Library:** A collection of self-help materials and educational resources to aid students in their personal and academic development.
- **Counsellors Dashboard:** Enables counsellors to manage their profiles, session availability, and interact with students.
- **Admin Panel:** Gives system administrators the tools they need to manage users and system content, which keeps things running smoothly.

The front end of the platform is created using React, the server is built with Node.js, and the database is developed with MongoDB. This makes sure that the platform is quick and can grow with its users.

### 3.1 Architectural Design

The Counsel-X platform's design is based on a modular framework, which means that multiple subsystems work together to make the whole system operate. The system is broken down into smaller parts, each of which is in charge of some task. This makes the system scalable, easy to maintain, and efficient.

The modules work together to make sure that students, counsellors, and administrators all have a smooth experience. The main parts of the system are:

**1. Frontend (User Interface):** The React-based frontend handles the user interface, allowing users (students, counsellors, administrators) to interact with the platform. It connects to the backend via APIs, providing updates in real time and a pleasant experience for users.

**2. Backend (Server-side):** The backend, which is built with Node.js, handles business logic, user authentication, and access to the database. It handles user requests including making appointments, keeping track of counsellors availability, and processing payments.

**3. Database:** The MongoDB database stores data related to users, sessions, resources, and payments. It ensures flexible data storage and fast retrieval, supporting the growth of the platform as more users join.

**4. Real-Time Communication:** The Socket.IO module enables students and counsellors talk to each other in real time during live sessions. This makes sure that signals are sent rapidly, which makes the environment more engaging.

**5. Payment Processing:** For safe and easy payment processing, the Stripe API is built in. It lets students pay for sessions online in a safe way.

The system is built on a client-server architecture, which means that the client side (frontend) talks to the server side (backend) to process input and give back response. This makes sure that the client and server don't get mixed up, with the client handling user interaction and the server handling data and business logic. The general design uses a multi-tiered architecture to make the system easy to scale, change, and keep up with. The subsystems are not closely connected, so modifications to one module won't effect the whole system. This makes it easy to add new features to the platform in the future.

### 3.1a System Architecture Diagram:

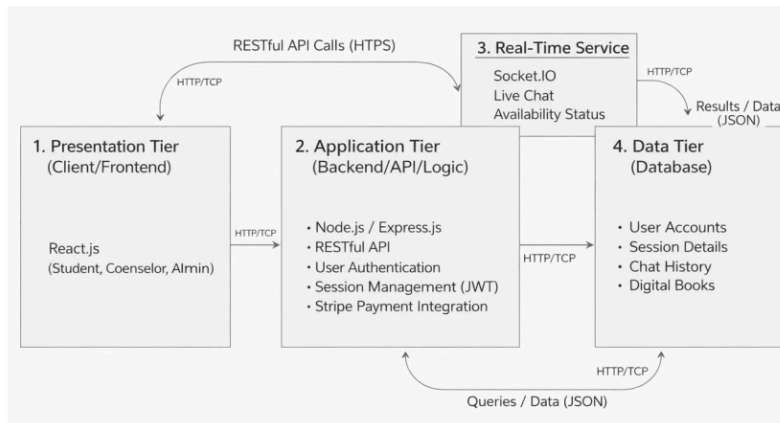


Figure 3.1a: Counsel-X System Architecture

### 3.2 Process flow/Representation:

#### 3.2a Session Booking Process:

The Session Booking Process allows students to choose a counsellors based on their availability and specialization. Students can select an available time slot for a counselling session, review the details, and proceed to secure payment through the Stripe API. Once the payment is confirmed, the system sends a session confirmation to both the student and counsellors, ensuring both parties are informed of the upcoming session.

**Session Booking Process Diagram:**

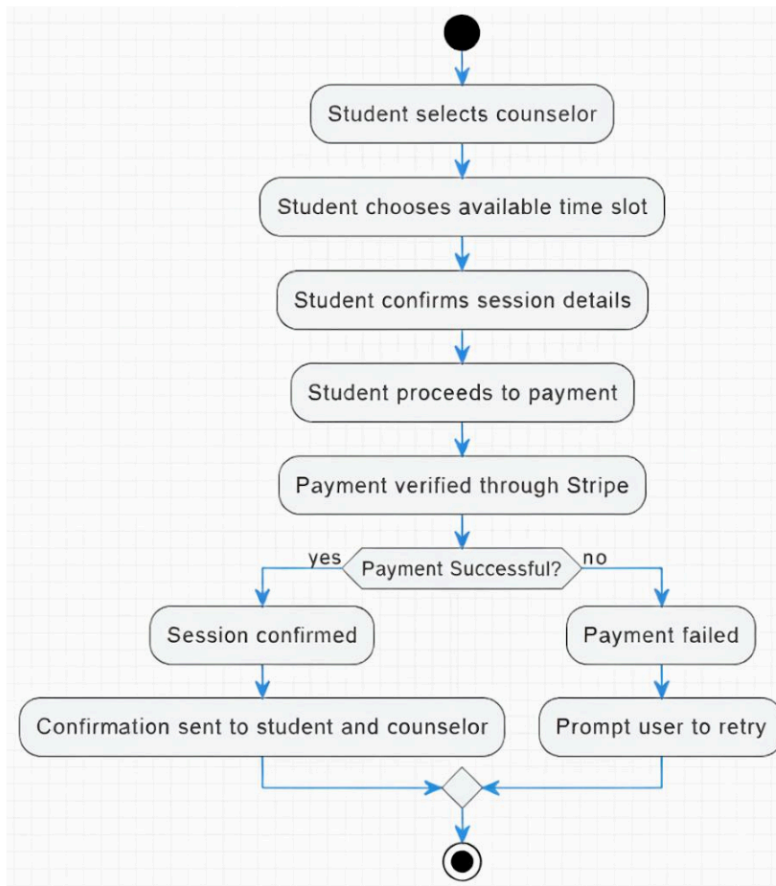


Figure 3.2a : Activity Diagram - Session Booking Process

### 3.2b Real-Time Communication Process:

The Real-Time Communication Process enables live, text-based interaction between students and counsellors during the scheduled session. Using Socket.IO, a secure connection is established, and the session takes place in real-time. The session has a 45-minute timer, and once the session ends, the chat history is stored for future reference, ensuring efficient communication and support.

#### Real-Time Communication Process Diagram:

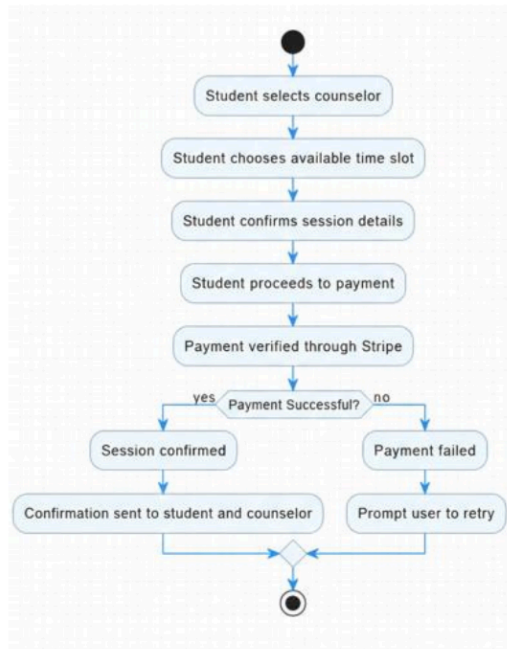


Figure 3.2b : Activity Diagram -Real Time Communication Process

### 3.2c Payment Processing Process:

The Payment Processing Process facilitates secure transactions for booked sessions. After selecting a counsellors and time slot, students enter their payment details, which are processed through Stripe. Upon successful payment, both the student and counsellors receive confirmation, and the session is officially booked. If the payment fails, the student is prompted to try again or use another payment method.

#### Payment Processing Process Diagram

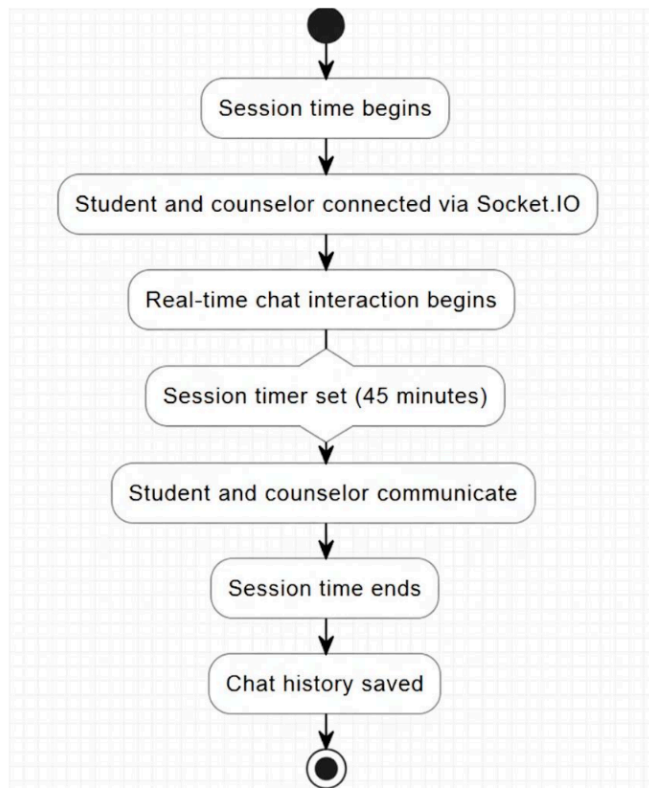


Figure 3.2c : Activity Diagram - Payment Processing Process Diagram

## 4. Design models

The design models for the Counsel-X system serve as blueprints that outline the system's architecture, data flow, and user interactions. These models help in visualizing the components of the system, their relationships, and the processes that drive the functionality. Key models such as the Class Diagram and Sequence Diagram are used to provide a structured approach to system design. They ensure that the system is scalable, user-friendly, and efficient in managing interactions between students, counsellors, and administrators.

### 4.1 Class Diagram

This model shows how the system is set up by showing the classes that are part of it, and their properties, methods, and connections. To understand how the main data entities, such as User, Session, Message, and Notification, work together in the system, this is very important.

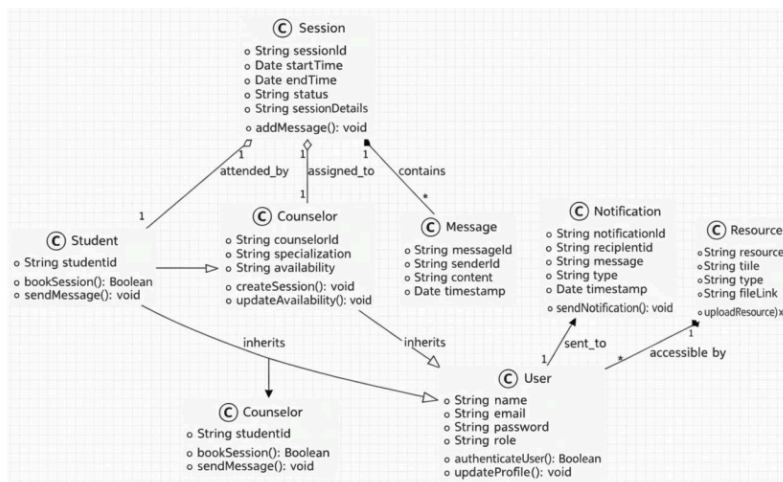


Figure 4.1: Counsel X Class Diagram

## 2 4.2 Sequence Diagram

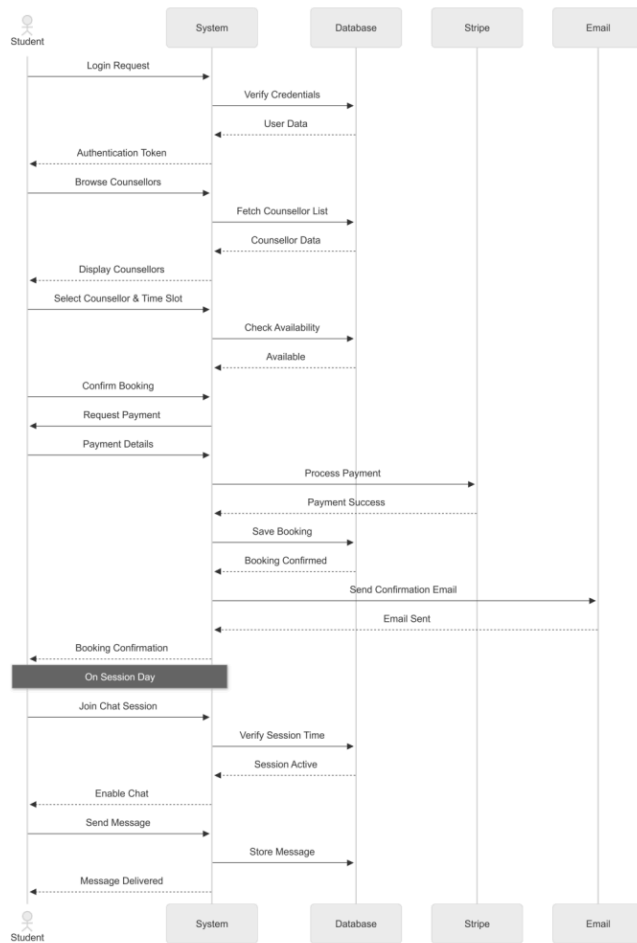


Figure 4.2: Sequence Diagram – Counsel X

### 4.3 Data Flow Diagram

#### DFD Level 0 - Context Diagram:

The Level-0 DFD (Context Diagram) for the Counsel-X system illustrates the high-level interactions between the system and its external entities: Students, Counsellors, and Administrators. It shows how data flows between these entities and the system for activities like session booking, real-time messaging, and user management. This diagram helps visualize the system's overall functionality and its environment.

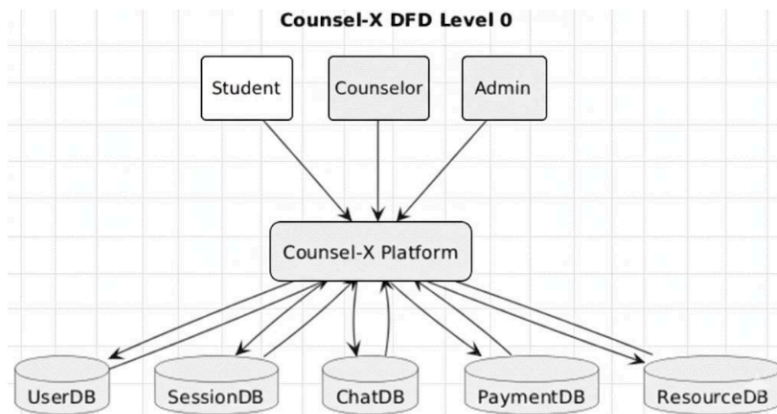
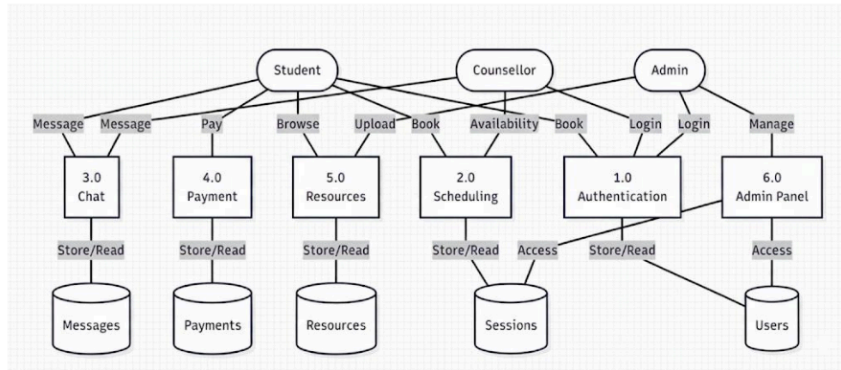


Figure 4.3: DFD Level 0 – Counsel X

**DFD Level 1:**

The Level-1 DFD for the Counsel-X system decomposes the main system into key processes: User Management, Session Management, Counsellors Availability, Messaging, Payment Processing, and Notification Management. It also shows the data stores for Users, Sessions, Messages, and Notifications, and how data flows between processes and stores. This diagram highlights the detailed interactions within the system, ensuring efficient management of sessions, messages, and user data.



**Figure 4.3: DFD Level 1 – Counsel X**

## 5. Data Design

Data design explains how the system's information domain is turned into data structures. It tells you how the system stores, processes, and organizes important data items. To manage more user and session data, the Counsel-X platform stores its data entities in a MongoDB NoSQL database. The data is organized into collections, including User, Session, Payment, Chat, and Counsellors Profile, each with its own set of details. This makes it easy for the system to grow and keeps the data safe.

### 5.1 Data Dictionary

User Entity:

| Attribute     | Data Type | Description                                       |
|---------------|-----------|---------------------------------------------------|
| _id           | Object Id | Unique identifier for the user                    |
| name          | String    | Full name of the user (2-50 characters)           |
| email         | String    | Email address used for login (unique)             |
| password      | String    | Hashed password using bcrypt                      |
| role          | String    | User role: student, counsellors, admin            |
| phone         | String    | Contact phone number                              |
| status        | String    | Account status: active, pending, suspended        |
| studentId     | String    | Student identification number (students only)     |
| feeStatus     | String    | Fee payment status: paid, partially_paid, pending |
| assignedBus   | ObjectId  | Reference to assigned bus (drivers only)          |
| assignedRoute | ObjectId  | Reference to assigned route (drivers only)        |
| licenseNumber | String    | Driver's license number (drivers only)            |

**Session Entity:**

| Attribute                 | Data Type                | Description                                          |
|---------------------------|--------------------------|------------------------------------------------------|
| <sup>7</sup><br><u>id</u> | ObjectId                 | Unique identifier for the session                    |
| studentId                 | String                   | ID of the student attending the session              |
| counsellorsId             | String                   | ID of the counsellors assigned to the session        |
| startTime                 | DateTime                 | Scheduled start time of the session                  |
| endTime                   | <sup>8</sup><br>DateTime | Scheduled end time of the session                    |
| status                    | String                   | Status of the session: pending, completed, cancelled |
| paymentStatus             | String                   | Payment status for the session: paid, pending        |

**Payment Entity:**

| Attribute                 | Data Type | Description                                        |
|---------------------------|-----------|----------------------------------------------------|
| <sup>6</sup><br><u>id</u> | ObjectId  | Unique identifier for the payment transaction      |
| sessionId                 | ObjectId  | ID of the associated session                       |
| amount                    | Decimal   | <sup>5</sup><br>Amount paid for the session        |
| paymentStatus             | String    | Status of the payment: successful, failed, pending |
| paymentMethod             | String    | Payment method used (e.g., Credit Card, PayPal)    |

**Chat Entity:**

| Attribute                  | Data Type | Description                                                |
|----------------------------|-----------|------------------------------------------------------------|
| <sup>10</sup><br><u>id</u> | ObjectId  | Unique identifier for the chat session                     |
| sessionId                  | ObjectId  | ID of the related session                                  |
| messages                   | List      | List of messages exchanged between student and counsellors |

|           |          |                                     |
|-----------|----------|-------------------------------------|
| timestamp | DateTime | Timestamp when the message was sent |
|-----------|----------|-------------------------------------|

#### Counsellors Profile Entity:

| Attribute      | Data Type | Description                                   |
|----------------|-----------|-----------------------------------------------|
| <u>id</u>      | ObjectId  | Unique identifier for the counsellors profile |
| counsellorsId  | String    | Unique ID for the counsellors                 |
| qualifications | String    | Counsellors's qualifications                  |
| specialization | String    | Area of expertise                             |
| availability   | List      | List of available time slots                  |
| rating         | Float     | Average rating from students                  |

## 6. Algorithm & Implementation

### 1. User Authentication

PROCEDURE authenticateUser(email, password)

BEGIN

    user ← findUserByEmail(email)

    IF user IS NULL THEN

        RETURN error("Invalid email or password")

    END IF

    isValid ← comparePassword(password, user.hashPassword)

    IF NOT isValid THEN

        RETURN error("Invalid email or password")

    END IF

    IF user.status = "suspended" THEN

        RETURN error("Account is suspended")

    END IF

    token ← generateJWT(user.\_id)

    RETURN success(user, token)

END

### 2. Schedule Counselling Session:

```

PROCEDURE scheduleSession(studentId, counsellorsId, sessionTime)
BEGIN
    student ← findUserById(studentId)
    IF student IS NULL THEN
        RETURN error("Student not found")
    END IF
    counsellors ← findUserById(counsellorsId)
    IF counsellors IS NULL THEN
        RETURN error("Counsellors not found")
    END IF

    availableSlots ← getAvailableSlots(counsellorsId)
    IF sessionTime NOT IN availableSlots THEN
        RETURN error("Counsellors not available at the selected time")
    END IF

    session ← createSession(studentId, counsellorsId, sessionTime)
    saveSession(session)
    createNotification(
        "Session Scheduled",
        "Your session with " + counsellors.name + " is scheduled for " + sessionTime,
        "student",
        studentId )
    createNotification(
        "Session Scheduled",
        "You have a new session with " + student.name + " scheduled for " + sessionTime,
        "counsellors",
        counsellorsId )
    RETURN success(session)
END

```

### 3. Start Real-Time Chat:

```

PROCEDURE startChat(sessionId)
BEGIN
    session ← findSessionById(sessionId)
    IF session IS NULL THEN
        RETURN error("Session not found")
    END IF
    IF session.status = "completed" THEN
        RETURN error("Session already completed")
    END IF
    localStream ← getUserMedia(audio)

```

```

peerConnection ← createPeerConnection()
callRef ← Firebase.createRoom(sessionId)
offer ← peerConnection.createOffer()
peerConnection.setLocalDescription(offer)
callRef.store("offer", offer)
listenForMessages()
RETURN success("Chat started")
END

```

#### 4. Send Message:

```

PROCEDURE sendMessage(sessionId, message)
BEGIN
    session ← findSessionById(sessionId)
    IF session IS NULL THEN
        RETURN error("Session not found")
    END IF
    IF session.status = "completed" THEN
        RETURN error("Session already completed")
    END IF
    callRef ← Firebase.ref(sessionId)
    callRef.store("message", message)

    RETURN success("Message sent")
END

```

#### 5. End Session:

```

PROCEDURE endSession(sessionId)
BEGIN
    session ← findSessionById(sessionId)
    IF session IS NULL THEN
        RETURN error("Session not found")
    END IF
    IF session.status = "completed" THEN
        RETURN error("Session already completed")
    END IF
    session.status ← "completed"
    session.endTime ← currentDate()
    saveSession(session)
    createNotification(
        "Session Completed",
        "Your session has been successfully completed",
        "student",
    )
END

```

```
        session.studentId )
createNotification(
    "Session Completed",
    "You have completed the session with " + session.studentName,
    "counsellors",
    session.counsellorsId
)
RETURN success("Session ended successfully")
END
```

#### **6. Send Poke Notification:**

```
PROCEDURE sendPoke(targetUserId)
BEGIN
    CALL cloudFunction("sendPokeNotification", { "recipientId": targetUserId })
    RETURN success("Poke notification sent")
END
```

## 7. Software Requirements Traceability Matrix

A mapping of functional requirements to the design components and implementation aspects that correspond to those needs is provided by the requirements monitoring matrix. This not only guarantees that all standards are totally covered, but it also makes it easier to do impact analysis for any future adjustments.

| Req. No | Requirement                            | Design Component   | Implementation                                 |
|---------|----------------------------------------|--------------------|------------------------------------------------|
| FR-1    | Manage User Accounts                   | User Module        | userController.manageAccounts()                |
| FR-2    | Maintain Book Library Data             | Library Module     | libraryController.updateBooks()                |
| FR-6    | Profile Creation and Management        | Profile Module     | profileController.createProfile()              |
| FR-7    | Course Purchase and Payment Processing | Payment Module     | paymentController.processPayment()             |
| FR-8    | Schedule Counselling Sessions          | Session Module     | sessionController.scheduleSession()            |
| FR-9    | Access Dashboard and Resources         | Dashboard Module   | dashboardController.loadDashboard()            |
| FR-12   | Profile Management                     | Profile Module     | profileController.manageProfile()              |
| FR-13   | Create and Manage Sessions             | Session Module     | sessionController.manageSessions()             |
| FR-14   | Real-Time Chat Interface               | Chat Module        | chatController.startChat()                     |
| FR-15   | Create Counselling                     | Counselling Module | counselling<br>Controller.createCounselling () |

### 7.1 Traceability Matrix:

| Req Number | Requirement        | Design Component    | Class / Function                          |
|------------|--------------------|---------------------|-------------------------------------------|
| FR-01      | User Registration  | AuthService         | registerUser(),<br>verifyAccount()        |
| FR-02      | User Login         | AuthService         | loginUser(),<br>validateCredentials()     |
| FR-06      | Real-Time Chat     | ChatService         | initializeChat(),<br>sendMessage()        |
| FR-08      | Session Scheduling | SessionService      | bookSession(),<br>viewAvailability()      |
| FR-10      | Session Management | SessionService      | approveSession(),<br>cancelSession()      |
| FR-12      | Payment Processing | PaymentService      | processPayment(),<br>verifyPayment()      |
| FR-13      | Resource Library   | LibraryService      | fetchResources(),<br>uploadMaterial()     |
| FR-14      | Notifications      | NotificationService | sendNotification(),<br>getNotifications() |
| FR-15      | Profile Management | ProfileService      | updateProfile(),<br>getProfile()          |

## 8. Human Interface Design

The system presented in the final report provides a web-based interface supporting three primary user roles: Student, Counsellors/Instructor, and Administrator. The interface follows a clean, structured, and user-friendly layout with clear navigation, consistent typography, and responsive elements. All screens are designed to support efficient task completion including profile operations, course purchasing, counselling session management, library access, and real-time chat communication.

### 8.1 Screen Images:

#### 8.1a Login Page:

The screenshot displays the login interface for a counseling website. At the top, a dark blue navigation bar contains the text 'StudentCounselor' and several links: 'Home', 'Counsellors', 'Service', 'About Us', 'Log In', and 'Book a session'. The 'Log In' and 'Book a session' links are highlighted in yellow. Below the navigation bar, the main content area is white and features a 'Login' form. The form has a title 'Login' and two input fields: 'Email' and 'Password'. Below these fields is a yellow 'Login' button. At the bottom of the form, there are two links: 'Forgot Password?' and 'Register'. The footer is a dark blue bar with the text 'Your Website Name' on the left, a vertical list of links (Home, About Us, Services, Admin) in the center, and 'Terms of Service' and 'Privacy Policy' on the right. At the very bottom, a copyright notice reads '© 2024 Your Company Name. All rights reserved.'

FIGURE 8.1a: Login of Counsel X

### 8.1b Register page:

The screenshot shows the 'Personal Information' registration form on the StudentCounselor website. The form includes input fields for Name, Email, Password, and Confirm Password, followed by a 'Log In' link and 'Previous' and 'Next' buttons. The website header features navigation links (Home, Counselors, Service, About Us) and buttons for 'Log In' and 'Book a session'. The footer contains the website name, contact information, and copyright notice.

StudentCounselor Home Counselors Service About Us Log In Book a session

Personal Information

Name

Email

Password

Confirm Password

Log In

Previous Next

Your Website Name

Home About Us Services Contact Us

Terms of Service Privacy Policy

© 2024 Your Company Name. All rights reserved.

FIGURE 8.1b: Register page of Counsel X

### 8.1c Dashboard Page:

The screenshot displays the dashboard for a student named Anas. It features a sidebar with navigation options like 'Website Name', 'COUNSELOR'S', 'Book Library', and 'LogOut'. The main content area shows the user's name, a chat session with a counselor named Soban, and a session start timer. The footer includes the website name, contact information, and copyright notice.

StudentCounselor Home User Counselors Service About Us Log Out

Website Name

COUNSELOR'S

Soban-Counselor sobanmuntax17@gmail.com

Book Library

LogOut

Your Website Name

Home About Us Services Contact Us

© 2024 Your Company Name. All rights reserved.

Dashboard: Anas-Student

Chat With: Soban-Counselor

Session Starts In: 0:05:59

Type a message... Send

Terms of Service Privacy Policy

FIGURE 8.1c: Dashboard of Counsel X

### 8.1d Front Page:

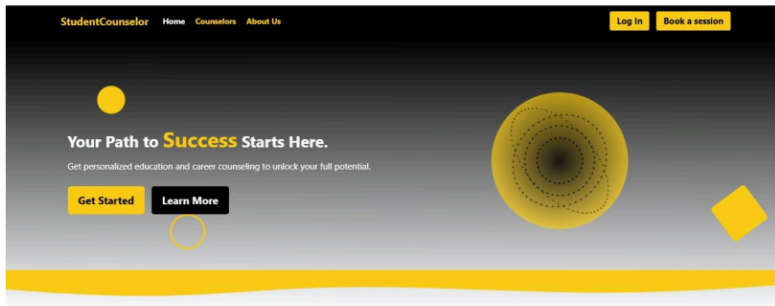


FIGURE 8.1d: Front Page of Counsel X

### 8.1e Registration Page:

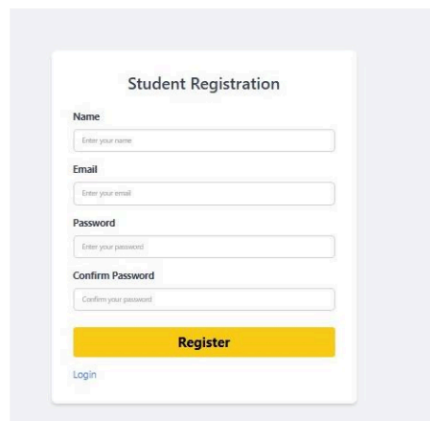
The image shows a "Student Registration" form. It includes fields for "Name", "Email", "Password", and "Confirm Password". Each field has a placeholder text: "Enter your name", "Enter your email", "Enter your password", and "Confirm your password". Below the fields is a yellow "Register" button and a blue "Login" link.

FIGURE 8.1e: Registration Page of Counsel X

**8.2 Screen Objects and Actions:**

| Screen Object      | Type              | Actions                                         |
|--------------------|-------------------|-------------------------------------------------|
| Navigation Bar     | Component         | Navigate pages, access profile, logout          |
| Login Form         | Form              | Enter credentials, authenticate                 |
| Registration Form  | Form              | Enter user details, select role, create account |
| Dashboard Cards    | Display           | Show counts, updates, quick links               |
| User Table         | Data Table        | View, edit, approve/reject users                |
| Library Table      | Data Table        | Browse and search books/materials               |
| Course List        | List Component    | View courses, open details, purchase items      |
| Session Scheduler  | Form/Panel        | Select date/time, request or approve sessions   |
| Chat Box           | Interactive Panel | Send/receive messages in real time              |
| Notification Panel | Panel             | View, mark read, delete notifications           |
| Create Modal       | Dialog            | Add new entries with validation                 |
| Edit Modal         | Dialog            | Modify content and save changes                 |
| Confirm Dialog     | Dialog            | Approve, reject, or delete actions              |
| Filter Controls    | Form Controls     | Apply filters to result lists                   |

## 9. Appendix

### References:

#### UML Diagrams & Modeling.

- Agile Modeling UML Artifacts Guide. Retrieved from: <http://agilemodeling.com/artifacts/>

#### Database Design & ER Diagrams

- Ullman, J. D. *The Complete Database Book*. Retrieved from: [http://people.inf.elte.hu/nikovits/DB2/Ullman\\_The\\_Complete\\_Book.pdf](http://people.inf.elte.hu/nikovits/DB2/Ullman_The_Complete_Book.pdf)

#### Data Flow Diagrams (DFD)

- Agile Modeling DFD Guidelines. Retrieved from: <http://agilemodeling.com/artifacts/dataFlowDiagram.htm>

#### Software Architecture & Design

- Sommerville, I. (2010). *Software Engineering* (9th Edition). Addison-Wesley Professional.
- Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley Professional.

#### Technical References

[11] MongoDB, Inc. (2024). *MongoDB Documentation*. Retrieved from: <https://www.mongodb.com/docs/>

[12] React Documentation. (2024). *React Official Documentation*. Retrieved from: <https://react.dev/learn>

[13] Node.js Foundation. (2024). *Node.js Documentation*. Retrieved from: <https://nodejs.org/docs/latest/api/>

[14] Stripe, Inc. (2024). *Stripe API Documentation*. Retrieved from: <https://docs.stripe.com/>

[15] Socket.IO. (2024). *Socket.IO Documentation*. Retrieved from: <https://socket.io/docs/v4>

[16] Express.js. (2024). *Express.js Documentation*. Retrieved from: <https://expressjs.com/>

[17] JSON Web Tokens. (2024). *JWT Introduction*. Retrieved from: <https://jwt.io/introduction>

## Plagiarism Report:

sdd a

### ORIGINALITY REPORT

**14%**

SIMILARITY INDEX

**6%**

INTERNET SOURCES

**2%**

PUBLICATIONS

**12%**

STUDENT PAPERS

### PRIMARY SOURCES

**1** Submitted to Higher Education Commission  
Pakistan  
Student Paper 3%

**2** www.coursehero.com  
Internet Source 2%

**3** Submitted to University of Wales Institute,  
Cardiff  
Student Paper 2%

**4** Submitted to Botswana Accountancy College  
Student Paper 1%

**5** Submitted to GL Bajaj Institute of Technology  
and Management  
Student Paper 1%

**6** Submitted to Asia Pacific International College  
Student Paper 1%

**7** Submitted to University of Houston Clear  
Lake  
Student Paper 1%

**8** pdfcoffee.com  
Internet Source 1%

**9** Submitted to NCC Education  
Student Paper 1%

**10** Submitted to Asia Pacific University College of  
Technology and Innovation (UCTI)  
Student Paper <1%

## ORIGINALITY REPORT

10%

SIMILARITY INDEX

6%

INTERNET SOURCES

0%

PUBLICATIONS

8%

STUDENT PAPERS

## PRIMARY SOURCES

|    |                                                                                            |     |
|----|--------------------------------------------------------------------------------------------|-----|
| 1  | Submitted to Higher Education Commission Pakistan<br>Student Paper                         | 2%  |
| 2  | www.coursehero.com<br>Internet Source                                                      | 2%  |
| 3  | Submitted to University of Wales Institute, Cardiff<br>Student Paper                       | 1%  |
| 4  | Submitted to University of Leicester<br>Student Paper                                      | 1%  |
| 5  | Submitted to Asia Pacific International College<br>Student Paper                           | 1%  |
| 6  | rps.wku.edu.et<br>Internet Source                                                          | 1%  |
| 7  | Submitted to NCC Education<br>Student Paper                                                | 1%  |
| 8  | Submitted to Regent Independent School and Sixth Form College<br>Student Paper             | <1% |
| 9  | ijarcce.com<br>Internet Source                                                             | <1% |
| 10 | Submitted to University of Wolverhampton<br>Student Paper                                  | <1% |
| 11 | Rohit Rastogi, Gautam Yadav, Nishant Kumar. "Intelligent analysis of flowers and knowledge | <1% |

generation: an empirical study for agriculture  
4.0", Walter de Gruyter GmbH, 2024

Publication

12

pdfcoffee.com  
Internet Source

<1 %

Exclude quotes On

Exclude matches Off

Exclude bibliography On